

University of Tehran
College of Engineering
School of Electrical and Computer Engineering

Computer Networks

Programming Project 4

TCP-Mini over IP/UDP and Congestion Control

Team Members

Shayan Maleki

Student ID: 810102515

Mani Hosseini

Student ID: 810102552

July 18, 2026

Table of Contents

1 Project Introduction	2
2 Phase 0: Initial Design and Code Structure	2
2.1 Goal of Phase 0	2
2.2 Project Files	2
2.3 Packet Header	3
2.4 Packet Types	3
2.5 Serialization and Deserialization	3
2.6 Checksum	4
2.7 File Reading and Writing	4
2.8 Phase 0 Result	4
3 Phase 1: Reliable Transfer with Stop-and-Wait	4
3.1 Goal of Phase 1	4
3.2 Stop-and-Wait Method	4
3.3 Receiver Behavior	5
3.4 Timeout and FIN Handling	5
3.5 Test Settings	5
3.6 Test Result Summary	6
3.7 Normal Sample Transfer	6
3.7.1 Sender Output	6
3.7.2 Receiver Output	6
3.7.3 Important Log Events	7
3.8 Timeout and Retransmission Test	7
3.8.1 Sender Output	7
3.8.2 Receiver Output	7
3.8.3 Important Sender Log Events	8
3.8.4 Important Receiver Log Events	8
3.9 Phase 1 Analysis	8
4 Phase 2: Sliding Window	9
4.1 Goal and Implementation	9
4.2 Phase 2 Settings	9
4.3 Sliding Window Sample Test	10
4.3.1 Sender Output	10
4.3.2 Receiver Output	10
4.3.3 Important Sender Log Events	10

4.4	Comparison with Phase 1	11
4.5	Phase 2 Analysis	12
4.6	Why Sliding Window Performs Better	12
5	Phase 3: Artificial Loss and Delay	13
5.1	Goal and Implementation	13
5.2	Test Settings	14
5.3	Measured Results	14
5.4	Sample Run with 10% Loss and 100 ms Delay	15
5.4.1	Sender Output	15
5.4.2	Receiver Output	15
5.4.3	Important Log Events	15
5.5	Effect of Increasing Delay	16
5.6	Why More Loss Causes More Retransmissions and Lower Through- put	16
5.7	Phase 3 Conclusion	17
6	Phase 4: Congestion Control	17
6.1	Goal and Implementation	17
6.2	Slow Start	17
6.3	Congestion Avoidance	18
6.4	Cumulative ACK Processing	18
6.5	Reaction to Timeout	18
6.6	Why the Congestion Window Is Reduced	19
6.7	Generated Result Files	19
6.8	Graph Generation	19
6.9	Normal Verification Configuration	20
6.10	Phase 4 Conclusion	20
7	Phase 5: Final Experiments	21
7.1	Experiment Setup	21
7.2	Final Results	21
7.3	Normal Network Result	22
7.4	Medium-Loss Result	23
7.5	High-Loss Result	25
7.6	Effect of Loss and Delay	26
7.7	Congestion-Control Analysis	27
7.8	Phase 5 Conclusion	27
8	Phase 6: Fast Retransmit and Fast Recovery (Bonus)	28
8.1	Goal and Implementation	28
8.2	Comparison Setup	28

8.3	Measured Results	29
8.4	Cases Where Fast Retransmit Occurred	29
8.5	Comparison of Timeout Count	29
8.6	Comparison of Transfer Time	29
8.7	Congestion Window Without Fast Retransmit	30
8.8	Congestion Window With Fast Retransmit	31
8.9	Phase 6 Conclusion	32

Project Introduction

The goal of this project is to make a small version of TCP in the application layer. UDP is used for sending and receiving packets. The reliable parts such as sequence number, ACK, timeout, retransmission, sliding window, and congestion control are made by our own code.

The sender reads the input file and divides it into packets. The receiver checks the packets and rebuilds the output file. The program works with text and binary files.

This report currently contains Phase 0 and Phase 1.

Phase 0: Initial Design and Code Structure

2.1 Goal of Phase 0

The goal of Phase 0 was to prepare the basic structure of the project. This phase includes packet design, serialization, deserialization, checksum, file reading, file writing, timer, logger, and Makefile.

2.2 Project Files

File	Use
<code>sender.cpp</code>	Reads the input file and creates DATA packets.
<code>receiver.cpp</code>	Receives packets and writes accepted data into the output file.
<code>packet.hpp</code>	Defines the header, packet types, serialization, deserialization, and checksum.
<code>timer.hpp</code>	Measures transfer time and timeout periods.
<code>logger.hpp</code>	Saves important program events in log files.
<code>Makefile</code>	Compiles the sender and receiver programs.

Table 1: Main project files

2.3 Packet Header

```
struct PacketHeader {
    uint32_t seq_num;
    uint32_t ack_num;
    uint16_t length;
    uint16_t checksum;
    uint8_t flags;
};
```

Field	Size	Use
seq_num	32 bits	Sequence number of a DATA or FIN packet.
ack_num	32 bits	Next sequence number expected by the receiver.
length	16 bits	Number of payload bytes in the packet.
checksum	16 bits	Detects an error in the header or payload.
flags	8 bits	Shows if the packet is DATA, ACK, or FIN.

Table 2: Packet header fields

The serialized header size is 13 bytes. The maximum payload size is 1400 bytes. The payload size used in Phase 1 was 1000 bytes.

2.4 Packet Types

- **DATA:** contains file data and a sequence number.
- **ACK:** tells the sender the next expected sequence number.
- **FIN:** tells the receiver that the transfer is finished.

DATA packets have payload data. ACK and FIN packets do not have payload data.

2.5 Serialization and Deserialization

Before sending a packet, the program changes its fields into bytes. Multi-byte values are changed to network byte order.

```
seq_num | ack_num | length | checksum | flags | payload
```

After receiving a packet, the program changes the bytes back into header fields and payload. It checks the packet size, payload length, packet flag, and checksum. If one check fails, the packet is rejected.

2.6 Checksum

The project uses a 16-bit one's complement checksum. During checksum creation, the checksum field is first set to zero. The packet bytes are added as 16-bit words, and the result is inverted.

The receiver calculates the checksum again. If the new value is not equal to the received value, the packet is not accepted.

2.7 File Reading and Writing

The sender opens the input file in binary mode and reads it in small chunks. The receiver also opens the output file in binary mode and writes only correct and expected DATA payloads.

Binary mode is needed because the project must work with both text and binary files.

2.8 Phase 0 Result

Phase 0 prepared the base of the project. Packet design, serialization, deserialization, checksum, file handling, timer, logger, and Makefile were completed. These parts were used in Phase 1.

Phase 1: Reliable Transfer with Stop-and-Wait

3.1 Goal of Phase 1

In Phase 1, reliable file transfer was added on top of UDP. The sender sends one packet and waits for its ACK. If the ACK does not arrive before the timeout, the same packet is sent again.

The output file must be exactly equal to the input file. The program also records sent packets, retransmissions, timeouts, and total transfer time.

3.2 Stop-and-Wait Method

For a DATA packet with sequence number N :

1. The sender sends DATA packet N .

2. The sender waits for ACK $N + 1$.
3. If ACK $N + 1$ arrives, the sender continues with the next packet.
4. If timeout happens, packet N is sent again.

Only one packet can wait for an ACK at one time.

3.3 Receiver Behavior

The receiver keeps the next expected sequence number.

- If the packet sequence number is equal to the expected value, its payload is written into the output file.
- If the sequence number is smaller, the packet is a duplicate and is not written again.
- If the sequence number is larger, the packet is out of order and is not written.
- For duplicate or out-of-order packets, the receiver sends the current ACK again.
- A packet with a wrong checksum is ignored.

3.4 Timeout and FIN Handling

The timeout value is 1000 ms. If the correct ACK is not received during this time, the sender sends the same packet again. Old ACKs and ACKs with a wrong number are ignored.

After all DATA packets are acknowledged, the sender sends a FIN packet. The receiver sends the final ACK and waits for 3000 ms. If FIN is received again, the receiver sends the same final ACK again.

3.5 Test Settings

Setting	Value
Receiver IP	127.0.0.1
Receiver port	8080
Payload size	1000 bytes
Timeout	1000 ms
FIN wait time	3000 ms

Table 3: Phase 1 test settings

3.6 Test Result Summary

Test	Bytes	DATA	Total TX	Retrans.	Timeouts	Time (ms)
Small text	16	1	2	0	0	0
Binary 2500	2500	3	4	0	0	1
Empty file	0	0	1	0	0	0
Binary 1 MiB	1048576	1049	1050	0	0	129
Forced timeout	20	1	5	3	3	3348

Table 4: Sender test results

File Correctness

All completed tests produced an output file exactly equal to the input file.

3.7 Normal Sample Transfer

The sample input was:

```
Hello TCP-Mini!
```

3.7.1 Sender Output

```
Transfer completed successfully
Input bytes: 16
DATA packets: 1
Total transmissions: 2
Retransmissions: 0
Timeouts: 0
Invalid ACKs: 0
Ignored ACKs: 0
Transfer time: 0 ms
Throughput: 0.158656 Mbps
```

3.7.2 Receiver Output

```
Receiver listening on port 8080
Transfer completed successfully
Output bytes: 16
Accepted packets: 1
Duplicate packets: 0
Out-of-order packets: 0
Invalid packets: 0
```

```
Ignored packets: 0
ACKs sent: 2
Transfer time: 0 ms
```

3.7.3 Important Log Events

```
elapsed_ms=0 event=PACKET_SENT type=DATA seq=0
elapsed_ms=0 event=ACK_RECEIVED ack=1 type=DATA
elapsed_ms=0 event=PACKET_SENT type=FIN seq=1
elapsed_ms=1 event=ACK_RECEIVED ack=2 type=FIN
```

Normal Test Result

The sender sent one DATA packet and one FIN packet. No timeout or retransmission happened. The output file was exactly equal to the input file.

3.8 Timeout and Retransmission Test

For this test, the receiver was paused for a few seconds. The sender did not receive the ACK and retransmitted the DATA packet three times.

3.8.1 Sender Output

```
Transfer completed successfully
Input bytes: 20
DATA packets: 1
Total transmissions: 5
Retransmissions: 3
Timeouts: 3
Invalid ACKs: 0
Ignored ACKs: 3
Transfer time: 3348 ms
Throughput: 4.77794e-05 Mbps
```

3.8.2 Receiver Output

```
Receiver listening on port 8080
Transfer completed successfully
Output bytes: 20
Accepted packets: 1
Duplicate packets: 3
Out-of-order packets: 0
Invalid packets: 0
Ignored packets: 0
```

```
ACKs sent: 5
Transfer time: 0 ms
```

3.8.3 Important Sender Log Events

```
elapsed_ms=0 event=PACKET_SENT type=DATA seq=0
elapsed_ms=1000 event=TIMEOUT type=DATA seq=0
elapsed_ms=1001 event=PACKET_RETRANSMITTED type=DATA seq=0 attempt=2
elapsed_ms=2001 event=TIMEOUT type=DATA seq=0
elapsed_ms=2001 event=PACKET_RETRANSMITTED type=DATA seq=0 attempt=3
elapsed_ms=3002 event=TIMEOUT type=DATA seq=0
elapsed_ms=3002 event=PACKET_RETRANSMITTED type=DATA seq=0 attempt=4
elapsed_ms=3348 event=ACK_RECEIVED ack=1 type=DATA
elapsed_ms=3348 event=PACKET_SENT type=FIN seq=1
elapsed_ms=3349 event=ACK_RECEIVED ack=2 type=FIN
```

3.8.4 Important Receiver Log Events

```
elapsed_ms=3608 event=DATA_ACCEPTED seq=0 bytes=20
elapsed_ms=3608 event=DATA_DUPLICATE seq=0
elapsed_ms=3608 event=DATA_DUPLICATE seq=0
elapsed_ms=3608 event=DATA_DUPLICATE seq=0
elapsed_ms=3608 event=FIN_ACCEPTED seq=1
```

Timeout Test Result

The sender had three timeouts and three retransmissions. The receiver accepted the first DATA packet and found three duplicate packets. The duplicate packets were not written again. The final output file was still exactly equal to the input file.

3.9 Phase 1 Analysis

The normal sample test had no timeout and no retransmission. The timeout test showed that the sender can send a packet again when an ACK does not arrive. It also showed that the receiver does not write duplicate data more than one time.

The results show that sequence numbers, ACK checking, timeout, retransmission, duplicate handling, checksum checking, and FIN handling work correctly.

Stop-and-Wait is reliable, but it sends only one packet before waiting. Phase 2 will improve this with a sliding window.

Phase 2: Sliding Window

4.1 Goal and Implementation

In Phase 2, Stop-and-Wait was changed to a fixed sliding window. The sender can send several DATA packets before waiting for acknowledgements. The selected window size was 5 packets.

The sender keeps two main sequence values:

- **base**: the sequence number of the oldest unacknowledged packet.
- **nextSequence**: the sequence number of the next packet that can be sent.

The sender continues sending while there is free space in the configured window. The receiver sends cumulative acknowledgements. An ACK value of N means that all packets before N were received and the next expected packet is N .

The receiver accepts only the packet with the expected sequence number. Duplicate packets are not written again. Future packets are counted as out of order. In both cases, the receiver sends the current cumulative ACK again.

If a timeout happens, the sender uses Go-Back-N and retransmits all unacknowledged packets. FIN is sent only after all DATA packets are acknowledged.

4.2 Phase 2 Settings

Table 5: Phase 2 settings

Setting	Value
Receiver IP	127.0.0.1
Receiver port	8080
Payload size	1000 bytes
Timeout	1000 ms
Window size	5 packets
FIN wait time	3000 ms

4.3 Sliding Window Sample Test

A 4500-byte sample was used to show one complete window. It produced five DATA packets because the payload size was 1000 bytes.

4.3.1 Sender Output

```
Transfer completed successfully
Input bytes: 4500
DATA packets: 5
Window size: 5
Total transmissions: 6
Retransmissions: 0
Timeouts: 0
Invalid ACKs: 0
Ignored ACKs: 0
Transfer time: 2 ms
Throughput: 12.3153 Mbps
```

4.3.2 Receiver Output

```
Receiver listening on port 8080
Transfer completed successfully
Output bytes: 4500
Accepted packets: 5
Duplicate packets: 0
Out-of-order packets: 0
Invalid packets: 0
Ignored packets: 0
ACKs sent: 6
Transfer time: 1 ms
```

4.3.3 Important Sender Log Events

```
elapsed_ms=1 event=PACKET_SENT seq=0
elapsed_ms=1 event=PACKET_SENT seq=1
elapsed_ms=1 event=PACKET_SENT seq=2
elapsed_ms=1 event=PACKET_SENT seq=3
elapsed_ms=2 event=PACKET_SENT seq=4
elapsed_ms=2 event=ACK_RECEIVED ack=1
elapsed_ms=2 event=WINDOW_MOVED base=1 next_sequence=5
elapsed_ms=2 event=ACK_RECEIVED ack=2
elapsed_ms=2 event=WINDOW_MOVED base=2 next_sequence=5
elapsed_ms=2 event=ACK_RECEIVED ack=3
elapsed_ms=2 event=WINDOW_MOVED base=3 next_sequence=5
elapsed_ms=2 event=ACK_RECEIVED ack=4
```

```

elapsed_ms=2 event=WINDOW_MOVED base=4 next_sequence=5
elapsed_ms=2 event=ACK_RECEIVED ack=5
elapsed_ms=2 event=WINDOW_MOVED base=5 next_sequence=5
elapsed_ms=2 event=FIN_SENT seq=5
elapsed_ms=3 event=FIN_ACK_RECEIVED ack=6

```

The log shows that packets 0 to 4 were sent before ACK processing. In Stop-and-Wait, only one packet is sent before the sender waits.

4.4 Comparison with Phase 1

Both phases were tested with the same 5 MiB binary file. The input size was 5,242,880 bytes. Both phases used a payload size of 1000 bytes and a timeout of 1000 ms. Each transfer contained 5243 DATA packets and one FIN packet. Each phase was tested five times.

Table 6: Measured transfer time and throughput

Run	Phase 1 Time (ms)	Phase 1 Throughput (Mbps)	Phase 2 Time (ms)	Phase 2 Throughput (Mbps)
1	964	43.4770	691	60.6162
2	894	46.9136	577	72.6478
3	777	53.9801	663	63.1704
4	896	46.8023	781	53.6686
5	1067	39.2813	684	61.3074
Average	919.6	46.0909	679.2	62.2821

Table 7: Final comparison of Phase 1 and Phase 2

Metric	Phase 1	Phase 2
Protocol	Stop-and-Wait	Sliding Window
Window size	1 packet	5 packets
Average transfer time	919.6 ms	679.2 ms
Average throughput	46.0909 Mbps	62.2821 Mbps
Total transmissions	5244	5244
Retransmissions	0	0
Timeouts	0	0
File correctness	Correct	Correct

Phase 2 reduced the average transfer time by 26.14% and increased the average throughput by 35.13%. No extra transmissions were needed in the normal tests, and both output files were exactly equal to the input file.

4.5 Phase 2 Analysis

The sliding window completed the same transfer with a lower average time and higher average throughput. The average transfer time decreased by 26.14%, and the average throughput increased by 35.13%. Based on throughput, Phase 2 was about 1.35 times faster than Phase 1.

No retransmission happened because the tests were performed on localhost without artificial packet loss. The improvement came from sending several packets without waiting after every packet.

The results show that cumulative ACK handling, window movement, in-order receiver behavior, reliable FIN handling, and exact file reconstruction work correctly. Phase 3 will add controlled packet loss and delay.

4.6 Why Sliding Window Performs Better

Stop-and-Wait sends only one DATA packet and then waits for its ACK. During this waiting time, the sender does not send any other packet. Therefore, part of the available network capacity remains unused.

For example, suppose sending one packet takes a very short time, but receiving its ACK takes several milliseconds. In Stop-and-Wait, the sender remains idle during most of this period. It can send only one packet during each round-trip time.

Sliding Window removes most of this idle time. With a window size of 5, the sender can send packets 0, 1, 2, 3, and 4 before waiting for their ACKs. While ACKs for the first packets are travelling back to the sender, other packets are already travelling toward the receiver.

This behavior is called pipelining. Several packets can be inside the network at the same time. As a result, the sender uses more of the available network capacity.

Cumulative ACKs also help the sender move the window efficiently. For example, ACK 5 means that packets 0 through 4 were received correctly. The sender can remove all these packets from the unacknowledged window and send new packets.

The main differences are:

- Stop-and-Wait sends one packet during each round-trip time.
- Sliding Window can send several packets during the same round-trip time.
- Stop-and-Wait frequently leaves the sender idle.
- Sliding Window keeps the network pipeline more active.
- Sliding Window usually gives lower transfer time and higher throughput.

The benefit becomes larger when network delay increases. With a large round-trip time, Stop-and-Wait must wait a long time after every packet. Sliding Window can continue sending until the window becomes full.

In our tests, Phase 1 had an average transfer time of 919.6 ms and an average throughput of 46.0909 Mbps. Phase 2 had an average transfer time of 679.2 ms and an average throughput of 62.2821 Mbps.

Therefore, Sliding Window reduced the average transfer time by 26.14

The tests were performed on localhost, where network delay is very small. For this reason, the improvement was limited. In Phase 3, artificial delay will be added, and the advantage of Sliding Window should become more visible.

Phase 3: Artificial Loss and Delay

5.1 Goal and Implementation

Phase 3 tests the Phase 2 sliding-window protocol under an unreliable network. Artificial packet loss and ACK delay were added to the receiver. The sender still uses a window size of 5, cumulative ACKs, a 1000 ms timeout, and Go-Back-N retransmission.

The receiver uses the constants `LOSS_PERCENT` and `ACK_DELAY_MS`. For every valid DATA or FIN packet, a random number decides whether the packet is ignored. A dropped packet is not written and no ACK is sent for it. Before sending a normal ACK, the receiver waits for the configured delay.

A fixed random seed of 12345 was used so the loss pattern could be repeated. The same 20,000-byte binary input file was used in all tests. It produced 20 DATA packets. Each of the 16 loss and delay combinations was run once, and every output was checked against the input file.

5.2 Test Settings

Table 8: Phase 3 test settings

Setting	Value
Input size	20,000 bytes
DATA packets	20
Payload size	1000 bytes
Window size	5 packets
Timeout	1000 ms
Loss values	0%, 5%, 10%, 20%
ACK delay values	0, 50, 100, 200 ms
Random seed	12345

5.3 Measured Results

Table 9: Results for all loss and delay combinations

Loss	Delay	Dropped	Retrans.	Timeouts	Time (ms)	Throughput (Mbps)
0%	0 ms	0	0	0	2	63.712700
0%	50 ms	0	0	0	1072	0.149248
0%	100 ms	0	0	0	2126	0.075231
0%	200 ms	0	0	0	4215	0.037951
5%	0 ms	3	7	2	2003	0.079873
5%	50 ms	3	7	2	3075	0.052023
5%	100 ms	3	7	2	4112	0.038907
5%	200 ms	3	7	2	6218	0.025731
10%	0 ms	3	7	2	2002	0.079906
10%	50 ms	3	7	2	3077	0.051997
10%	100 ms	3	7	2	4117	0.038860
10%	200 ms	3	7	2	6217	0.025736
20%	0 ms	7	15	3	3005	0.053240
20%	50 ms	7	15	3	4085	0.039166
20%	100 ms	7	15	3	5120	0.031249
20%	200 ms	7	15	3	7218	0.022165

All 16 output files were exactly equal to the input file. The SHA-256 value used for verification was:

The 5% and 10% tests produced the same number of dropped packets in this experiment. This happened because a fixed random seed and a small file were used. The generated

random values caused the same packets to fall inside both loss limits. The 20% tests dropped more packets and showed the expected increase in retransmissions and timeouts.

5.4 Sample Run with 10% Loss and 100 ms Delay

5.4.1 Sender Output

```
Transfer completed successfully
Input bytes: 20000
DATA packets: 20
Window size: 5
Total transmissions: 28
Retransmissions: 7
Timeouts: 2
Invalid ACKs: 0
Ignored ACKs: 4
Transfer time: 4117 ms
Throughput: 0.0388598 Mbps
```

5.4.2 Receiver Output

```
Receiver listening on port 8080
Configured loss: 10%
Configured ACK delay: 100 ms
Transfer completed successfully
Output bytes: 20000
Accepted packets: 20
Duplicate packets: 0
Out-of-order packets: 4
Invalid packets: 0
Ignored packets: 0
Simulated dropped packets: 3
Delayed ACKs: 25
ACKs sent: 25
Transfer time: 4016 ms
```

5.4.3 Important Log Events

```
event=PACKET_DROPPED type=DATA seq=5
event=DATA_OUT_OF_ORDER seq=6 next_expected=5
event=DATA_OUT_OF_ORDER seq=7 next_expected=5
event=DATA_OUT_OF_ORDER seq=8 next_expected=5
event=DATA_OUT_OF_ORDER seq=9 next_expected=5
event=TIMEOUT base=5 next_sequence=10
event=PACKET_RETRANSMITTED seq=5
event=PACKET_RETRANSMITTED seq=6
```

```
event=PACKET_RETRANSMITTED seq=7
event=PACKET_RETRANSMITTED seq=8
event=PACKET_RETRANSMITTED seq=9
event=PACKET_DROPPED type=DATA seq=18
event=PACKET_DROPPED type=DATA seq=19
event=TIMEOUT base=18 next_sequence=20
event=PACKET_RETRANSMITTED seq=18
event=PACKET_RETRANSMITTED seq=19
```

Packet 5 was dropped, so packets 6 to 9 arrived while the receiver was still expecting packet 5. These packets were counted as out of order. After the timeout, the sender retransmitted the complete unacknowledged window. Later, packets 18 and 19 were dropped and were sent again after another timeout.

5.5 Effect of Increasing Delay

With 0% packet loss, increasing ACK delay from 0 ms to 200 ms increased the transfer time from 2 ms to 4215 ms. Throughput decreased from 63.712700 Mbps to 0.037951 Mbps.

The sender can send several packets in one window, but the window can move only after ACKs arrive. When every ACK is delayed, the sender waits longer before it can send new packets. Therefore, larger ACK delay increases the total transfer time and decreases throughput.

5.6 Why More Loss Causes More Retransmissions and Lower Throughput

When a DATA packet is dropped, the receiver does not move its expected sequence number. Later packets in the same window become out of order and cannot be written. The sender receives no new cumulative ACK for the missing packet and finally reaches the timeout.

After a timeout, Go-Back-N retransmits the missing packet and all later unacknowledged packets in the window. Therefore, one dropped packet can cause several retransmissions. As the loss percentage increases, packet drops happen more often, which creates more timeouts and more repeated transmissions.

Retransmitted packets use network capacity but do not add new file data. The sender also remains idle while waiting for timeout. These two effects increase transfer time and reduce useful throughput. In the 0 ms delay tests, increasing loss from 0% to 20% increased retransmissions from 0 to 15 and reduced throughput from 63.712700 Mbps to 0.053240 Mbps.

5.7 Phase 3 Conclusion

The protocol completed every tested transfer correctly under all 16 combinations. Increasing ACK delay increased transfer time because the sender window moved more slowly. Increasing packet loss produced more timeouts and Go-Back-N retransmissions. Even with 20% configured loss and 200 ms ACK delay, the final output file was exactly equal to the input file.

Phase 4: Congestion Control

6.1 Goal and Implementation

The goal of Phase 4 was to add TCP-like congestion control to the sliding-window sender. In the previous phases, the sender used a fixed window. In this phase, the sender uses a congestion window called `cwnd`. The value of `cwnd` changes after successful ACKs and timeout events.

The sender also keeps a fixed receiver window. The real sending window is the smaller value between the receiver window and the congestion window.

```
effectiveWindow = min(  
    RECEIVER_WINDOW,  
    max(1, static_cast<int>(cwnd))  
);
```

The initial congestion window is one packet, and the initial Slow Start threshold is 16 packets.

```
double cwnd = 1.0;  
double ssthresh = 16.0;
```

The fixed receiver window is 64 packets. The payload size is 1000 bytes, and the timeout is 1000 ms.

6.2 Slow Start

Slow Start is active while `cwnd` is smaller than `ssthresh`. For every newly acknowledged packet, the sender increases `cwnd` by one.

```
if (cwnd < ssthresh) {  
    cwnd += 1.0;
```

```
}
```

During one complete window, several ACKs may arrive. Therefore, the congestion window grows approximately as follows:

```
1, 2, 4, 8, 16
```

This fast growth allows the sender to find a useful sending rate without starting with a large number of packets.

6.3 Congestion Avoidance

When `cwnd` becomes equal to or larger than `ssthresh`, the sender enters Congestion Avoidance. In this state, the congestion window grows more slowly.

```
cwnd += 1.0 / cwnd;
```

After approximately one complete window of ACKs, `cwnd` increases by about one packet. Slow Start gives fast growth, while Congestion Avoidance increases the sending rate more carefully.

6.4 Cumulative ACK Processing

The receiver still sends cumulative ACKs. If the old sender base is 10 and ACK 14 arrives, packets 10, 11, 12, and 13 were newly acknowledged.

```
newlyAcknowledged = ackNumber - base;
```

The sender updates `cwnd` once for every newly acknowledged packet. This is necessary because one cumulative ACK may acknowledge several DATA packets.

6.5 Reaction to Timeout

A timeout is treated as a possible sign of congestion. When a timeout happens, the sender reduces `ssthresh` and resets `cwnd`.

```
ssthresh = max(2.0, cwnd / 2.0);  
cwnd = 1.0;
```

The sender returns to Slow Start and retransmits the unacknowledged packets with Go-Back-N. The important timeout events are:

```
TIMEOUT  
SSTHRESH_UPDATED  
CWND_RESET  
PACKET_RETRANSMITTED
```

6.6 Why the Congestion Window Is Reduced

A timeout means that the sender did not receive a useful ACK before the timer expired. One possible reason is that too many packets entered the network and some packets were lost.

If the sender continues with the same large congestion window, more packets may be lost. Resetting `cwnd` lowers the sending rate and gives the network time to recover. The sender then increases the rate again with Slow Start and Congestion Avoidance.

This behavior makes the protocol adaptive. Successful ACKs increase the sending rate, while timeout events decrease it.

6.7 Generated Result Files

The sender creates the following files:

- `results/phase4/events.log`: records ACK, window update, timeout, retransmission, state change, and completion events.
- `results/phase4/cwnd.csv`: records `cwnd`, `ssthresh`, the congestion state, the sender base, and the next sequence number over time.
- `results/phase4/throughput.csv`: records acknowledged bytes, interval throughput, and average throughput over time.

The columns of `cwnd.csv` are:

```
elapsed_ms,cwnd,ssthresh,state,event,base,next_sequence
```

The columns of `throughput.csv` are:

```
elapsed_ms,acknowledged_bytes,interval_mbps,average_mbps
```

6.8 Graph Generation

The Python program `plot_results.py` reads the CSV files and creates these graphs:

```
results/phase4/plots/cwnd_time.png  
results/phase4/plots/throughput_time.png
```

The first graph shows `cwnd` and `ssthresh` over time. The second graph shows interval throughput and average throughput over time. The actual graphs and detailed performance tests are presented in Phase 5.

6.9 Normal Verification Configuration

For a normal verification run, artificial packet loss and ACK delay are disabled.

```
const int LOSS_PERCENT = 0;
const int ACK_DELAY_MS = 0;
```

The expected normal behavior is:

- `cwnd` starts from one packet.
- Slow Start increases `cwnd` quickly.
- The sender enters Congestion Avoidance after reaching `ssthresh`.
- No timeout reset should happen during a normal local transfer.
- The output file should be exactly equal to the input file.
- The event log, CSV files, and graph files should be created successfully.

The actual performance measurements for different loss and delay conditions are left for Phase 5.

6.10 Phase 4 Conclusion

Phase 4 added adaptive congestion control to the reliable UDP protocol. Slow Start increases the sending rate quickly at the beginning. Congestion Avoidance continues with slower growth after the threshold is reached. A timeout reduces `ssthresh`, resets `cwnd`, and starts Slow Start again.

The sender also records congestion-window changes and throughput values. These files are used to create and analyze the required graphs in Phase 5.

Phase 5: Final Experiments

7.1 Experiment Setup

The final phase evaluated congestion control under three network conditions. The same 200,000-byte binary file was used in every experiment. The payload size was 1000 bytes, so each transfer contained 200 DATA packets and one FIN packet. The timeout was 1000 ms, the receiver window was 64 packets, the initial value of `cwnd` was 1, and the initial value of `ssthresh` was 16.

The three required configurations were:

- Normal network: 0% loss and 0 ms ACK delay.
- Medium-loss network: 10% loss and 50 ms ACK delay.
- High-loss network: 20% loss and 100 ms ACK delay.

Every transfer used the same input file and fixed random seed. File correctness was checked with a byte comparison. The input and were equal in all three experiments.

7.2 Final Results

Test	Loss	Delay	Total TX	Retrans.	Timeouts	Time (ms)	Throughput	Match
Normal	0%	0 ms	201	0	0	23	68.746045 Mbps	Yes
Medium	10%	50 ms	311	110	27	37233	0.042972 Mbps	Yes
High	20%	100 ms	336	135	41	61318	0.026093 Mbps	Yes

Table 10: Phase 5 sender results

Test	Dropped	Out of Order	Final cwnd	Final ssthresh
Normal	0	0	25.028944	16.000000
Medium	31	79	3.000000	2.787777
High	62	73	4.997533	2.000000

Table 11: Phase 5 receiver and congestion-control results

All three output files were exactly equal to the common 200,000-byte input file. The protocol completed the transfer even with 20% artificial loss and 100 ms ACK delay.

7.3 Normal Network Result

The normal experiment had no artificial loss and no ACK delay. The sender transmitted 200 DATA packets and one FIN packet. There were no retransmissions and no timeouts. The complete transfer took 23 ms and the measured throughput was 68.746045 Mbps.

The congestion window began at 1 packet. Slow Start increased it quickly until it reached `sssthresh` at about 2.7 ms. After that point, Congestion Avoidance increased it more slowly. The final congestion window was 25.028944 packets. Because no timeout occurred, `sssthresh` remained equal to 16.

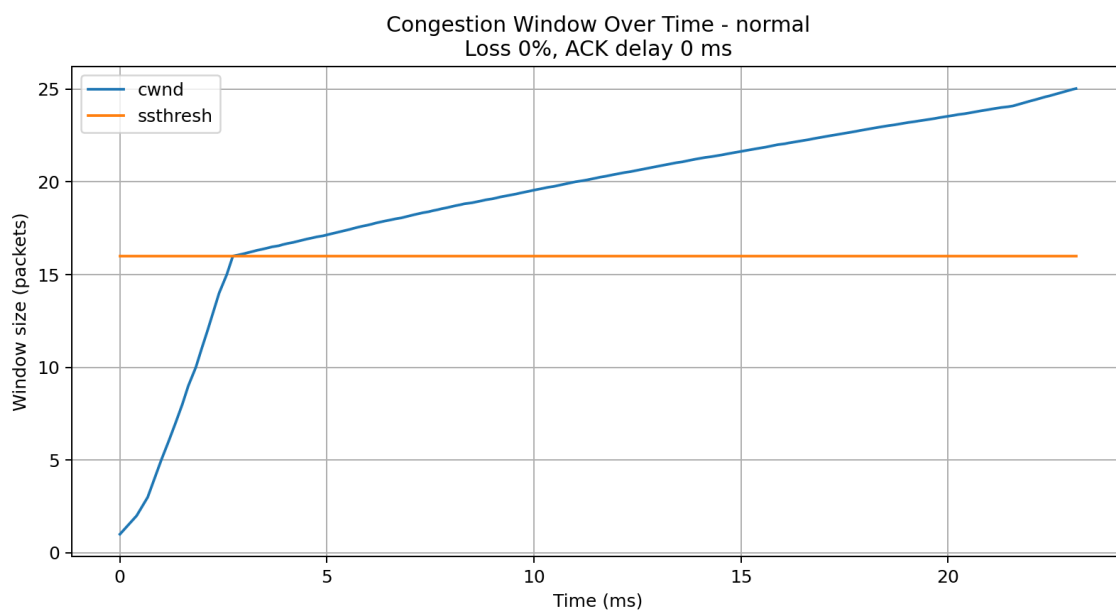


Figure 1: Congestion window over time in the normal experiment

The throughput graph shows that average throughput increased as the congestion window grew. The interval throughput changed between ACK events, while the average value became more stable near the end of the short local transfer.

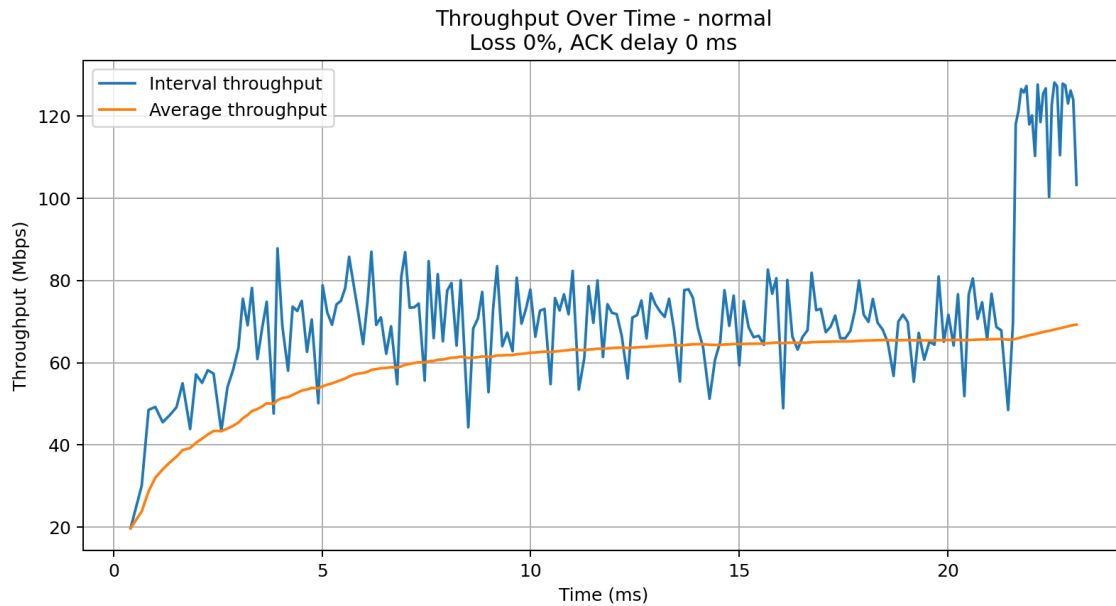


Figure 2: Throughput over time in the normal experiment

7.4 Medium-Loss Result

The medium experiment used 10% loss and 50 ms ACK delay. The receiver dropped 31 packets. The sender performed 110 retransmissions and experienced 27 timeouts. The transfer took 37233 ms and the final throughput was 0.042972 Mbps.

The congestion-window graph contains 27 timeout-reset points. At every timeout, `cwnd` was reset to 1 and `ssthresh` was reduced. The congestion window repeatedly grew to approximately 3 to 7.6 packets and then fell back to 1. It did not reach the original threshold of 16 during this experiment.

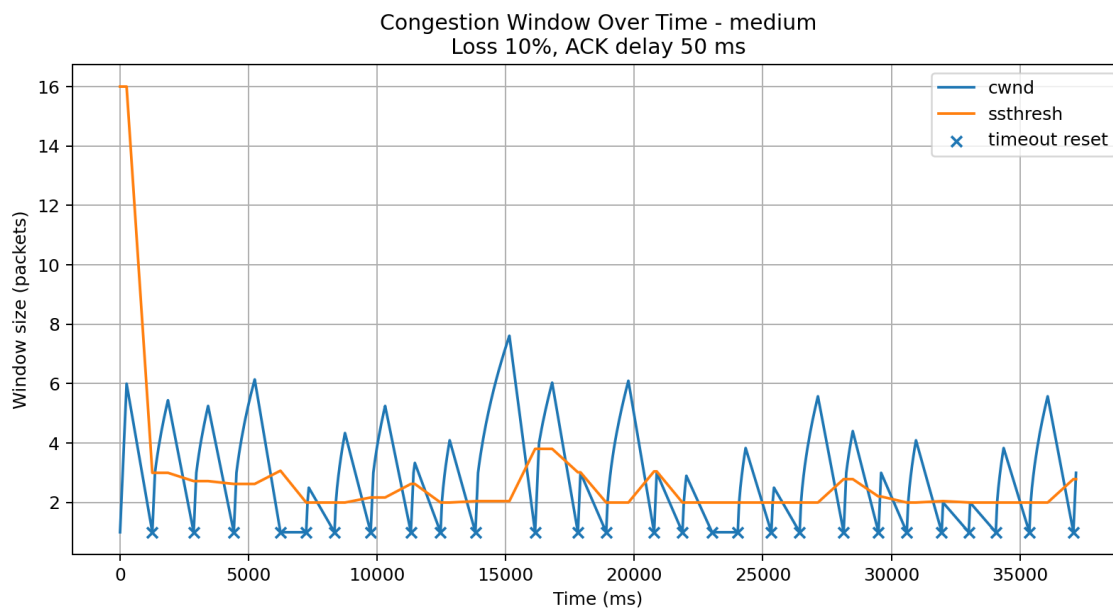


Figure 3: Congestion window over time with 10% loss and 50 ms delay

The throughput graph has repeated drops during timeout periods. The average throughput gradually settled near 0.043 Mbps. ACK delay slowed every window update, while packet loss caused retransmission and one-second timeout waiting periods.

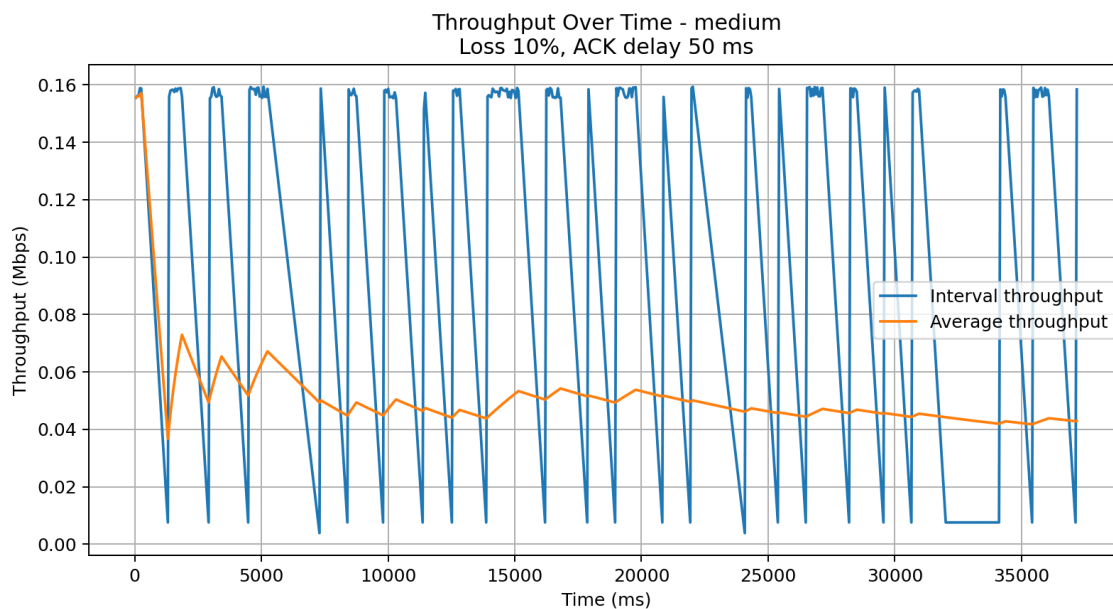


Figure 4: Throughput over time with 10% loss and 50 ms delay

7.5 High-Loss Result

The high-loss experiment used 20% loss and 100 ms ACK delay. The receiver dropped 62 packets. The sender performed 135 retransmissions and experienced 41 timeouts. The transfer took 61318 ms and the final throughput was 0.026093 Mbps.

The complete file was still transferred correctly. However, the congestion window remained small for most of the experiment. It was reset to 1 at 41 timeout points and reached a maximum of only about 6.2 packets. The final `ssthresh` was 2, showing that repeated timeout reactions reduced the allowed growth.

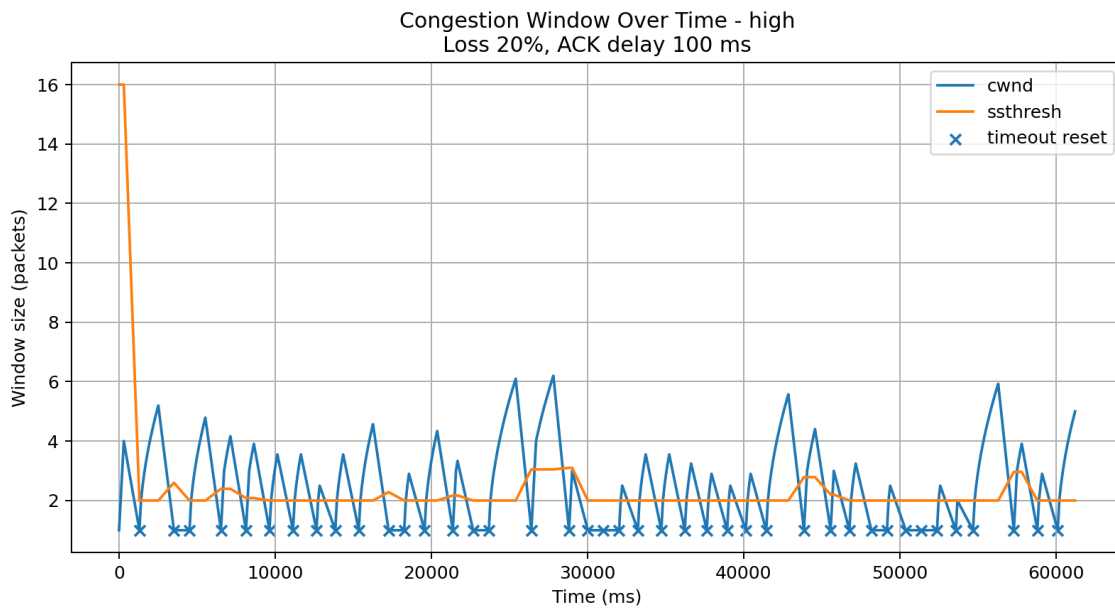


Figure 5: Congestion window over time with 20% loss and 100 ms delay

The throughput graph shows longer low-throughput periods than the medium experiment. The final throughput was 0.026093 Mbps, which was 39.28% lower than the medium-loss result. The total transfer time was 64.69% higher than the medium-loss result.

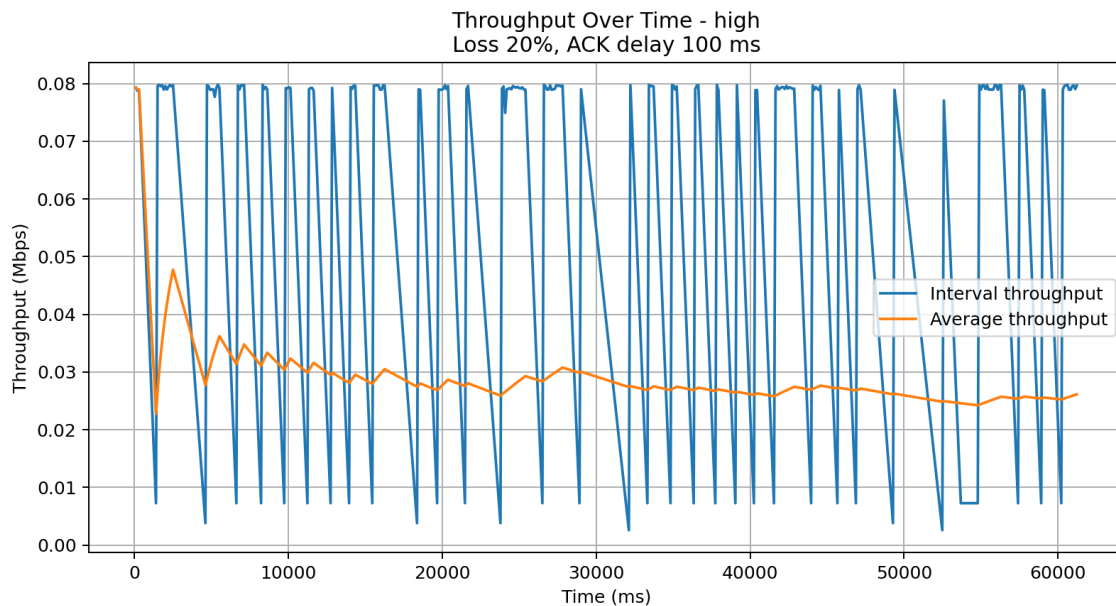


Figure 6: Throughput over time with 20% loss and 100 ms delay

7.6 Effect of Loss and Delay

Increasing loss and delay reduced protocol performance in several connected ways:

- Dropped packets increased from 0 to 31 and then to 62.
- Retransmissions increased from 0 to 110 and then to 135.
- Timeouts increased from 0 to 27 and then to 41.
- Transfer time increased from 23 ms to 37233 ms and then to 61318 ms.
- Throughput decreased from 68.746045 Mbps to 0.042972 Mbps and then to 0.026093 Mbps.

Loss prevents the receiver from advancing its cumulative ACK. Later packets may arrive out of order, and the sender eventually reaches its timeout. Go-Back-N then retransmits all unacknowledged packets. These repeated transmissions do not add new file data, but they consume time and network capacity.

ACK delay also slows window movement because the sender must wait longer before learning that packets were accepted. In this implementation, the one-second timeout is especially expensive. Repeated loss therefore creates long idle periods, resets `cwnd` to 1, and keeps the sender in small-window operation.

The high-loss test had twice as many dropped packets as the medium test. It also had 14 more timeouts and 25 more retransmissions. As a result, the high-loss transfer took about 1.65 times as long as the medium transfer.

7.7 Congestion-Control Analysis

The normal graph clearly shows the two congestion-control states. Slow Start produces fast growth from 1 to 16, and Congestion Avoidance produces slower growth above 16.

The medium and high graphs show repeated saw-tooth behavior. Successful ACKs increase `cwnd`, but each timeout resets it to 1 and reduces `ssthresh`. With higher loss and delay, the sender has less time to grow before another timeout occurs. Therefore, the average congestion window and useful throughput remain low.

The experiment also shows why TCP-style congestion control reduces the sending window after timeout. Continuing with a large window after loss would send more packets into an already unreliable path. Resetting `cwnd` lowers the sending rate and lets the sender search again for a safe rate.

7.8 Phase 5 Conclusion

All required experiments were completed with the exact same input file. The normal network produced fast growth, no retransmission, no timeout, and high throughput. The medium network produced repeated congestion-window reductions and much lower throughput. The high-loss network produced the most timeouts and retransmissions, but the protocol still reconstructed the complete file correctly.

The final results confirm that the implementation provides reliable file transfer, cumulative acknowledgements, Go-Back-N retransmission, Slow Start, Congestion Avoidance, timeout recovery, metric collection, and graph generation over UDP.

Phase 6: Fast Retransmit and Fast Recovery (Bonus)

8.1 Goal and Implementation

The bonus phase adds Fast Retransmit and Fast Recovery to TCP-Mini. In the previous implementation, a missing packet was normally retransmitted only after the 1000 ms timeout. In Phase 6, the sender also counts duplicate cumulative ACKs.

If the same ACK number is received three times, the ACK number identifies the first missing packet. The sender retransmits that packet immediately instead of waiting for timeout.

```
if (duplicateAckCount == 3) {
    ssthresh = max(2.0, cwnd / 2.0);
    retransmit(packets[base]);
}
```

Fast Recovery was also implemented. After Fast Retransmit, the congestion window becomes `ssthresh + 3`. Every additional duplicate ACK increases `cwnd` by one. When a new cumulative ACK arrives, the sender leaves Fast Recovery and sets `cwnd` to `ssthresh`. A real timeout still resets `cwnd` to one.

The sender records these new events:

```
DUPLICATE_ACK_RECEIVED
DUPLICATE_ACK_COUNT
FAST_RETRANSMIT
FAST_RECOVERY_ENTER
FAST_RECOVERY_DUP_ACK
FAST_RECOVERY_EXIT
```

8.2 Comparison Setup

The same 200,000-byte input file was transferred twice. Both runs used 10% artificial loss, 50 ms ACK delay, a 1000 ms timeout, receiver window 64, initial `cwnd` equal to 1, and initial `ssthresh` equal to 16.

The first run disabled Fast Retransmit and Fast Recovery. The second run enabled both mechanisms. The input and output files were compared byte by byte after each run.

8.3 Measured Results

Mode	Total TX	Retrans.	Fast Retrans.	Timeouts	Time (ms)	Throughput	Match
Disabled	311	110	0	27	37298	0.042897 Mbps	Yes
Enabled	395	194	15	25	43100	0.037123 Mbps	Yes

Table 12: Phase 6 comparison with and without Fast Retransmit

Phase 6 File Correctness

Both transfers reconstructed the complete 200,000-byte file correctly. Fast Retransmit and Fast Recovery did not change reliability.

8.4 Cases Where Fast Retransmit Occurred

Fast Retransmit occurred 15 times. The missing sequence numbers recorded in the event log were:

```
5, 27, 42, 50, 69, 71, 103, 114, 114,
118, 125, 127, 132, 141, 183
```

For example, the first event happened after three duplicate ACKs with ACK number 5. The sender concluded that packet 5 was missing and retransmitted it at approximately 411 ms, before the normal timeout could expire.

```
event=FAST_RETRANSMIT ack=5 missing_seq=5
old_cwnd=6.000000 new_cwnd=6.000000
old_ssthresh=16.000000 new_ssthresh=3.000000
```

Sequence number 114 triggered Fast Retransmit twice because the packet was still not successfully accepted after an earlier recovery attempt.

8.5 Comparison of Timeout Count

Without Fast Retransmit, the sender experienced 27 timeouts. With Fast Retransmit and Fast Recovery, it experienced 25 timeouts. Therefore, duplicate ACK detection prevented two timeout events, which is a reduction of approximately 7.41%.

This result confirms the main purpose of Fast Retransmit: some missing packets can be retransmitted from duplicate ACK evidence before the timeout expires.

8.6 Comparison of Transfer Time

The transfer without Fast Retransmit took 37298 ms. The transfer with Fast Retransmit and Fast Recovery took 43100 ms. In this particular experiment, enabling the bonus

mechanism increased transfer time by approximately 15.56% and reduced throughput from 0.042897 Mbps to 0.037123 Mbps.

The reason is visible in the measurements. The enabled run sent 395 packets instead of 311 and performed 194 retransmissions instead of 110. Fast Retransmit injected additional packets into the simulated lossy receiver. Because loss is applied to every received transmission, the enabled run produced 41 simulated drops instead of 31. The extra retransmissions reduced two timeouts, but the additional traffic and ACK delays were more expensive in this specific deterministic run.

Therefore, Fast Retransmit successfully reduced timeout waiting, but it did not guarantee a shorter transfer in this simplified Go-Back-N simulation. Its benefit depends on the loss pattern, the current window size, the delay, and the retransmission strategy.

8.7 Congestion Window Without Fast Retransmit

The disabled graph contains only timeout-reset points. Missing packets are recovered after the timeout, so `cwnd` repeatedly falls to one.

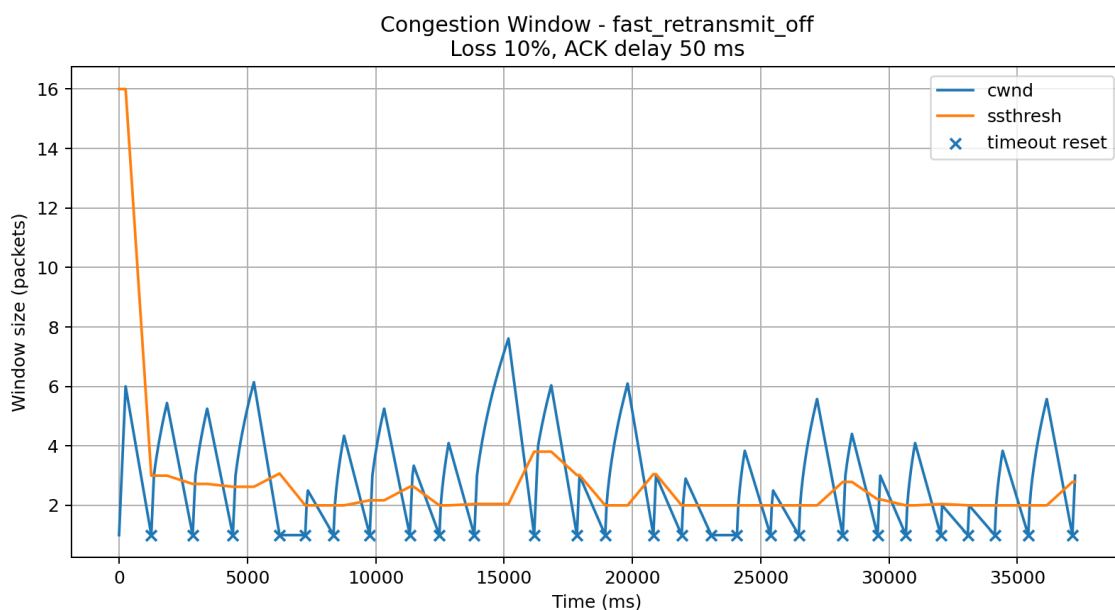


Figure 7: Congestion window with Fast Retransmit disabled

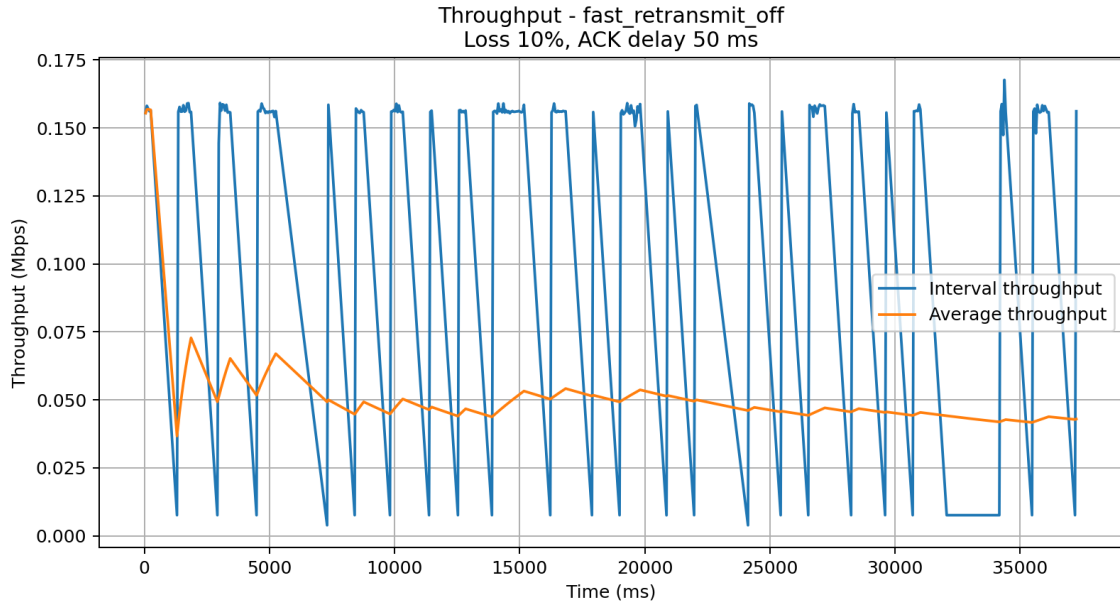


Figure 8: Throughput with Fast Retransmit disabled

8.8 Congestion Window With Fast Retransmit

In the enabled graph, triangle markers show the 15 Fast Retransmit points. Cross markers show timeout resets. At a Fast Retransmit point, `ssthresh` is reduced and the sender enters Fast Recovery instead of immediately resetting `cwnd` to one. When a new ACK arrives, the sender exits Fast Recovery at `ssthresh`.

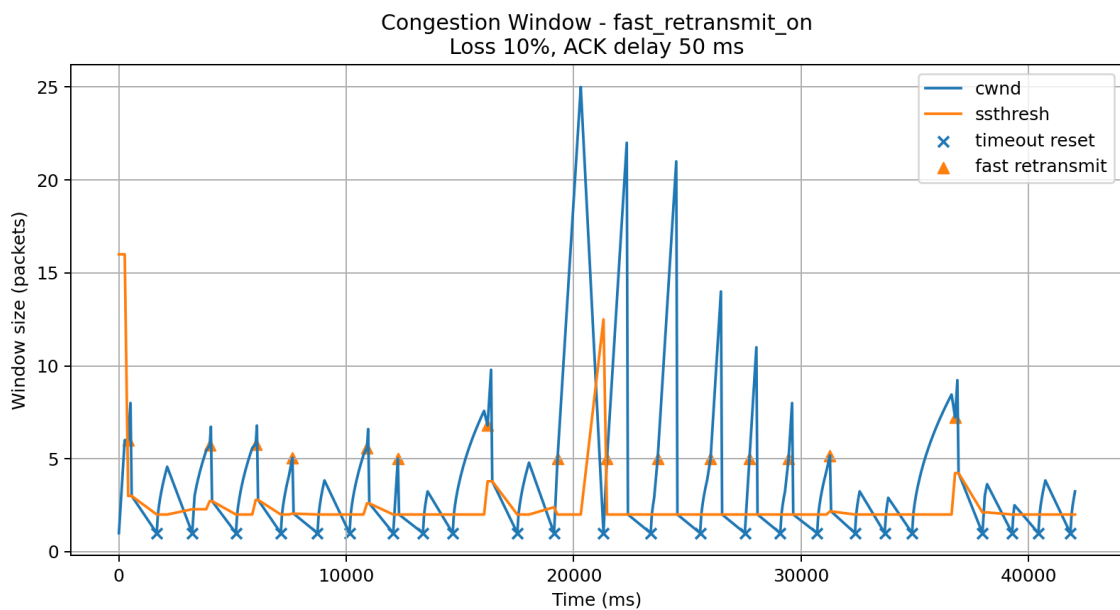


Figure 9: Congestion window with Fast Retransmit points marked

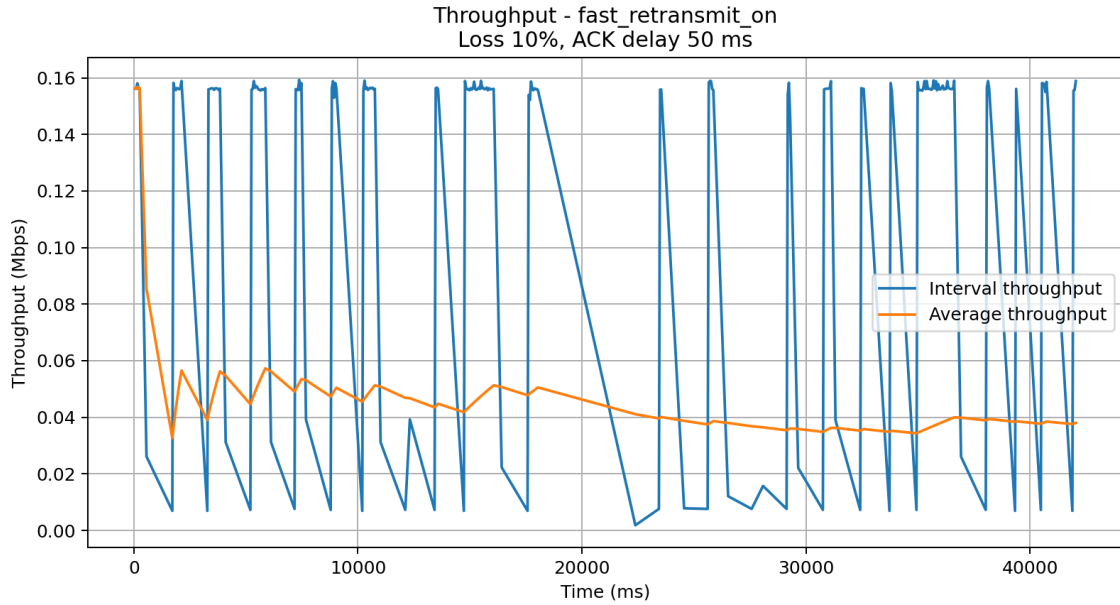


Figure 10: Throughput with Fast Retransmit and Fast Recovery enabled

8.9 Phase 6 Conclusion

Phase 6 successfully implemented duplicate ACK counting, Fast Retransmit, and Fast Recovery. Three duplicate ACKs caused immediate retransmission of the missing packet, and all Fast Retransmit events were recorded and marked on the congestion-window graph.

The enabled run contained 15 Fast Retransmit events and reduced the timeout count from 27 to 25. Both output files were exactly equal to the input file. In this test, extra retransmission traffic caused a longer total transfer time, showing that Fast Retransmit reduces timeout dependence but does not always improve performance in a simplified protocol and artificial-loss model.